Studio Nx

JS 285 — 클립 저장 이어 주기

2026-10-19

JavaScript 285 — 클립 저장 이어 주기 (`sequencer.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer.js`
작업	S1-10a 클립·연결 저장: 클립 생성/복제/이동/크기 조절/트리밍/슬립/자르기/삭제 및 헬퍼 함수 · 패턴 첫 생성 (FR-2196 클립·패턴, FR-2197 연결)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 284 (관련), 다음 286 `sequencer.js` 패턴·롤 노트 저장

1. 하는 일

타임라인에서 바뀐 클립을 서버에 바로 이어 저장합니다. `handleChange` 의 `create`·`duplicate`·`move`·`resize`·`trim`·`slip`·`slice`·`delete` 케이스를 통해, MIDI/오디오 클립 각각에 맞게 `POST /tracks/{id}/clips`, `PATCH /clips/{id}`, `DELETE /clips/{id}`, 그리고 필요할 때 `POST /patterns` 를 부릅니다.

```
편곡에서 클립을 놓거나 움직이거나 잘라내기
        ↓
sequencer-arrange.js → onChange({ type: ..., clip, trackId, start, length, right, ... })
        ↓
sequencer.js handleChange
  ├ create · duplicate → saveClipCreate        (오디오: sampleId 필요 / MIDI: 패턴 없으면 POST /pattern
s)
  ├ move · resize · trim · slip → saveClipUpdate (server-numbered clips only: has data-clip-id)
  ├ slice → saveClipUpdate(left) + saveClipCreate(right)
  └ delete → saveClipDelete
        ↓
sequencer-devices.js / 서버
        ↓
markDirty('clip') · clipWarn
```

2. 언제 쓰나

쓰는 곳	
`sequencer-arrange.js` 편곡 드래그	클립을 생성/옮김/크기조절/트리밍/슬립/자르기/삭제할 때 `onChange` 로 알립니다.
`sequencer.js` `handleChange`	위 타입들을 받아 서버에 이어 저장합니다.
`initPatternCommands`/다른 경로	`saveNewPattern` 은 `pattern.new` 명령에서도 씁니다.
디버그/에러	`clipWarn`/`errText` 가 저장 실패를 짧게 보여 줍니다.

3. 코드

3.1 handleChange 의 클립 케이스 (S1-10a)

```
    // 클립 (S1-10a FR-2196) — 화면에서 바뀐 클립을 서버에 이어 저장합니다
    case 'create':
```

```
    case 'duplicate':
      saveClipCreate(change);
      break;
    case 'move':
    case 'resize':
    case 'trim':
    case 'slip':
      saveClipUpdate(change);
      break;
    case 'slice':
      saveClipUpdate(clipGeometry(change.clip));
      saveClipCreate({ clip: change.right, trackId: change.trackId, start: styleNumber(change.right, '--
s'), length: styleNumber(change.right, '--l') });
      break;
    case 'delete':
      saveClipDelete(change);
      break;
```

3.2 헬퍼 함수

```
/** 마디 → tick (1박 = 960, 한 마디 = TICKS_PER_BAR) */
function toTicks(bars) {
  return Math.round((Number(bars) || 0) * TICKS_PER_BAR);
}

/** 클립 요소의 CSS 값(마디) 읽기 — --s 시작 · --l 길이 · --o 오프셋 */
function styleNumber(clip, name) {
  const v = parseFloat(clip.style.getPropertyValue(name));
  return isFinite(v) ? v : null;
}

/** 클립 요소의 지금 위치·길이·오프셋 (마디) */
function clipGeometry(clip) {
  return { clip: clip, start: styleNumber(clip, '--s'), length: styleNumber(clip, '--l'), offset: styleN
umber(clip, '--o') };
}

/** 클립 요소의 이름 (앞의 目/⇥ 글자 지우기) */
function clipElementName(clip) {
  const el = clip.querySelector('.sq-clip__name');
  return el ? el.textContent.replace(/^[目⇥]\s*/, '').trim() : '';
}

/** 클립 요소가 오디오인가 */
function isAudioClip(clip) {
  return clip.classList.contains('sq-clip--audio');
}
```

3.3 saveClipCreate

```
/**
 * 새 클립을 서버에 저장 (FR-2196) — 타임라인에 놓은 클립 · 복제 · 가른 오른쪽 토막.
 * MIDI 클립에 패턴이 없으면(빈 곳에 새로 그을 때) 패턴을 먼저 만들고, 다음 그리기가 재사용하도록
 * Draw 템플릿에 그 패턴을 담습니다.
 * @param {{clip: HTMLElement, trackId: number|string, start: number, length: number}} change
```

```
 */
async function saveClipCreate(change) {
  const clip = change.clip;
  const trackId = change.trackId;
  if (!clip || !trackId || !projectIdOf()) return;
  const audio = isAudioClip(clip);
  const body = {
    clip_type: audio ? 'AUDIO' : 'MIDI',
    name: clipElementName(clip).slice(0, 100) || null,
    start_tick: toTicks(change.start),
    length_tick: Math.max(1, toTicks(change.length)),
    offset_tick: toTicks(styleNumber(clip, '--o') || 0),
  };
  if (audio) {
    const sampleId = clip.dataset.sampleId ? Number(clip.dataset.sampleId) : null;
    if (!sampleId) { showToast('Pick a sample from the browser first'); clip.remove(); return; }
    body.sample_id = sampleId;
  } else {
    let patternId = clip.dataset.patternId ? Number(clip.dataset.patternId) : null;
    if (!patternId) {
      try {
        const pattern = await api.post('/patterns', {
          project_id: Number(projectIdOf()),
          name: body.name || 'Pattern',
          length_tick: body.length_tick,
        });
        patternId = pattern.pattern_id;
        clip.dataset.patternId = String(patternId);
        const tpl = getDrawTemplate();
        setDrawTemplate({ kind: 'midi', name: body.name || tpl.name, length: change.length, patternId: p
atternId });
      } catch (err) { clipWarn(err); return; }
    }
    body.pattern_id = patternId;
  }
  try {
    const saved = await api.post('/tracks/' + encodeURIComponent(String(trackId)) + '/clips', body);
    clip.dataset.clipId = String(saved.clip_id);
    markDirty('clip');
  } catch (err) { clipWarn(err); }
}
```

3.4 saveClipUpdate

```
/**
 * 클립의 위치·길이·오프셋을 서버에 저장 (FR-2196) — 옮기기 · 길이 조절 · 트리밍 · 내용 밀기.
 * 아직 서버에 없는 클립(번호 없음)이나 데모 클립은 건너뜁니다.
 * @param {{clip: HTMLElement, start?: number, length?: number, offset?: number}} change
 */
async function saveClipUpdate(change) {
  const clip = change.clip;
  if (!clip || !projectIdOf()) return;
  const id = clip.dataset.clipId;
  if (!id) return;
  const body = {};
  if (change.start != null) body.start_tick = toTicks(change.start);
```

```
  if (change.length != null) body.length_tick = Math.max(1, toTicks(change.length));
  if (change.offset != null) body.offset_tick = toTicks(change.offset);
  if (!Object.keys(body).length) return;
  try {
    await api.patch('/clips/' + encodeURIComponent(id), body);
    markDirty('clip');
  } catch (err) { clipWarn(err); }
}
```

3.5 saveClipDelete

```
/** 클립 지우기를 서버에 반영 (FR-2196) */
async function saveClipDelete(change) {
  const clip = change.clip;
  if (!clip || !projectIdOf()) return;
  const id = clip.dataset.clipId;
  if (!id) return;
  try {
    await api.delete('/clips/' + encodeURIComponent(id));
    markDirty('clip');
  } catch (err) { clipWarn(err); }
}
```

3.6 에러 알림

```
/** 클립·패턴 저장 실패 알림 (조용한 실패 방지) */
function clipWarn(err) {
  if (err instanceof ApiError && err.status === 401) return;
  showToast(errText(err));
  if (globalThis.console) globalThis.console.warn('[sequencer] clip', err);
}

/** 오류 → 짧은 글자 */
function errText(err) {
  if (err && err.status === 403) return 'You cannot change this project';
  if (err && err.status === 404) return 'That clip or pattern no longer exists';
  return 'Could not save the change — we will try again';
}
```

4. 왜 이렇게

선택	이유
`TICKS_PER_BAR` 를 써서 tick 으로 바꿈	DB·서버는 tick 을 쓰고, 프런트 편곡은 마디 단위(`--s`, `--l`, `--o`)를 씁니다. `PPQ=960` 기준과 일관되게 변환합니다.
MIDI 클립에 `patternId` 가 없으면 `POST /patterns` 먼저	빈 곳에 MIDI 클립을 그으면 곧장 패턴이 필요합니다. 패턴을 미리 만들고 `clip.dataset.patternId`/Draw 템플릿에 저장해 재사용과 롤 노트 저장이 이어지게 합니다.
오디오 클립은 `sampleId` 없으면 경고 후 제거	샘플을 고르지 않은 상태에서 오디오 클립을 놓는 건 서버로 보낼 수 없습니다. 화면상에 남겨두지 않기 위해 제거합니다.
`saveClipUpdate` 는 `data-clip-id` 가 있을 때만	서버에서 온 클립만 PATCH/DELETE 합니다. 데모(예시 화면) 클립은 서버 번호가 없으므로 건드리지 않습니다.

선택	이유
`slice` 는 왼쪽은 업데이트, 오른쪽은 생성	자른 뒤 왼쪽 조각은 원래 클립의 위치·길이만 바뀌므로 `PATCH`, 오른쪽 조각은 새 클립이므로 `POST` 로 이어 저장합니다.
`clipWarn` 이 401 은 무시	`api.js` 의 토큰 재발급 흐름에 맡기기 위해 토스트를 띄우지 않습니다.
offset 도 tick 으로 보냄	`offset_tick` 은 서버가 클립 내부 시작을 가리키는 값입니다. 화면의 `--o`(마디)를 tick 으로 변환해 보냅니다.

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 에서 다음을 확인합니다.

- 빈 타임라인에 MIDI 클립을 놓으면 `POST /patterns` → `POST /tracks/{id}/clips` 순으로 이어지고, 클립에 `data-pattern-id` 와 `data-clip-id` 가 생깁니다.
- 오디오 클립을 놓을 때 샘플을 안 고르면 토스트가 나오고 클립이 제거됩니다.
- 클립을 옮기면 `PATCH /clips/{id}` 로 `start_tick` 이 갱신됩니다.
- 클립을 늘리거나 트리밍/슬립하면 길이나 `offset_tick` 이 `PATCH` 로 보냅니다.
- 클립을 자르면 왼쪽은 `PATCH`, 오른쪽은 `POST` 로 보내집니다.
- 클립을 지우면 `DELETE /clips/{id}` 가 호출되고 화면에서 사라집니다.
- 복제한 클립도 `POST /tracks/{id}/clips` 로 새로 저장됩니다.
- 서버에서 불러온 클립(`data-clip-id` 있는 클립)만 업데이트/삭제됩니다.
- 테스트 14/14 통과. 스크린샷: `docs/screenshots/2026-10-19_s1-10a_clips_connections/`.